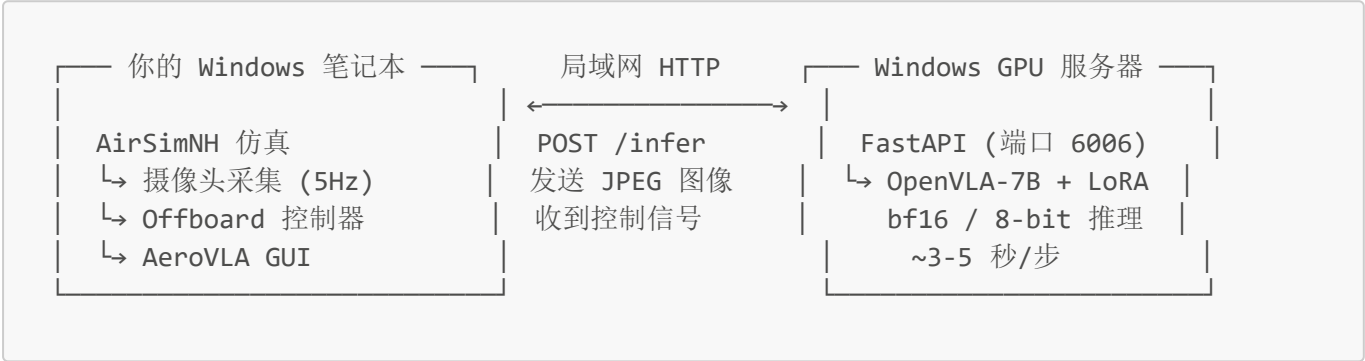


AeroVLA 局域网部署完全指南（笔记本 + Windows GPU 服务器）

Windows 笔记本运行 AirSim 仿真 + GUI，局域网内的 **Windows GPU 服务器** 运行 VLA 模型推理，两者通过 HTTP 直连，实现端到端无人机自主飞行。

架构说明



局域网延迟极低（<1ms），通信效率远优于远程云服务器。

1. 服务器到手：第一件事

1.1 确认硬件和系统

登录 Windows Server 后，打开 **PowerShell (管理员)**，执行：

```
nvidia-smi          # GPU 型号和显存大小
# 记下显存大小，这是选量化策略的核心依据

Get-PSDrive C | Select Used,Free # C 盘空闲空间（需≥50GB）
systeminfo | findstr "Memory"   # 系统内存（建议≥32GB）
python --version              # Python 版本（需 3.9~3.11）
nvcc --version                # CUDA 版本（需 11.8+）
```

1.2 根据显存决定量化策略

显存	对应显卡	推荐策略	精度	备注
≥24 GB	4090 / A100 / A6000	全精度 bf16	最佳	理想配置，零坑
16-23 GB	3090 / 4080	全精度 bf16	最佳	绕过 bnb
12-15 GB	4070 / 3060 12G / 3080	8-bit（谨慎）	可用	bnb 可能不稳定
8-11 GB	4060 Ti 8G / 3070	⚠️ 不建议	—	bnb 在 RTX 40 系列易卡死

你的服务器是 RTX 4090 (24GB)，这是本项目的理想配置。 无需 bitsandbytes、无需量化、无需 WSL2。全精度 bf16 直接加载，~14.5GB 显存占用，剩余 ~9.5GB 充足余量。

如果你用的是 12GB 以下的 RTX 40 系显卡（如 4070/4060），才需要考虑 [第 12 章 WSL 方案](#)。

1.3 确认服务器 IP

```
ipconfig
# 找到局域网 IPv4 地址，如 192.168.1.100
```

记下服务器的**局域网 IP**，后续所有连接都会用到。

1.4 确认 Windows 笔记本能与服务器互通

在笔记本 PowerShell 中：

```
ping 192.168.1.100
# 应看到正常回复
```

2. 安装 Python 环境

2.1 安装 Python（如果没有）

```
# 方式1: winget
winget install Python.Python.3.10

# 方式2: 手动下载
# https://www.python.org/downloads/ → 勾选 "Add to PATH"
```

2.2 确认 CUDA 和 PyTorch

```
# 如果 CUDA 未安装，从 NVIDIA 官网下载安装
# https://developer.nvidia.com/cuda-downloads → 选 Windows

# 装 PyTorch (CUDA 12.1 版)
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121

# 验证
python -c "import torch; print('torch:', torch.__version__, 'CUDA:',
torch.cuda.is_available())"
```

2.3 创建工程目录

```
mkdir C:\vla-server  
cd C:\vla-server
```

3. 上传代码到服务器

3.1 需要上传的文件（2 个）

文件	用途
aerovla_server.py	FastAPI 推理服务入口
aerovla_inference.py	VLA 推理引擎核心

3.2 上传方式

方式 A：SCP（最直接）

```
# 在笔记本 PowerShell 中执行  
cd d:\vla飞控部署  
scp aerovla_server.py aerovla_inference.py 管理员@192.168.1.100:C:/vla-server/
```

方式 B：局域网共享文件夹

1. 在 Windows Server 上创建共享文件夹 C:\vla-server\
2. 右键 → 属性 → 共享 → 添加 Everyone（读/写）
3. 笔记本上：\\192.168.1.100\vla-server
4. 直接拖拽文件进去

方式 C：U 盘 / 移动硬盘

直接拷贝过去。

3.3 验证上传

```
# 在 Windows Server 上  
dir C:\vla-server\aerovla_*.py  
# 应看到两个文件
```

4. 安装 Python 依赖

这是最容易出错的步骤，版本号必须严格匹配。

在 Windows Server 上打开 **PowerShell（管理员）**：

```
cd C:\vla-server
```

```
# 1. 核心依赖（版本号一个都不能错！）
```

```
pip install `
    "transformers==4.42.4" `
    "huggingface_hub==0.23.4" `
    "peft==0.11.1" `
    "timm>=0.9.10,<1.0.0" `
    "einops>=0.6.0" `
    "accelerate>=0.20.0"
```

```
# 2. bitsandbytes – 你的 4090 不需要装
```

```
# 全精度 bf16 直接跑，避开 bnb 所有兼容问题
```

```
# 如果是 12GB 以下显卡才需要装：
```

```
# pip install "bitsandbytes>=0.41.0"
```

```
# 3. Web 服务
```

```
pip install `
    "fastapi>=0.104.0" `
    "uvicorn>=0.24.0" `
    "python-multipart>=0.0.6"
```

```
# 4. 图像处理
```

```
pip install opencv-python-headless Pillow "numpy<2.0.0" scipy
```

```
# 5. JWT 认证
```

```
pip install "python-jose[cryptography]" bcrypt
```

```
# 6. 地勤服务（如果需要在同一台机器跑）
```

```
pip install sqlalchemy aiosqlite
```

注意：bitsandbytes 在 Windows 上编译安装可能较慢，如果报错 `No module named 'bitsandbytes'`，检查是否安装成功：

```
python -c "import bitsandbytes; print(bitsandbytes.__version__)"
```

4.1 验证安装

```
python -c "
import torch, transformers, peft, fastapi, cv2
print('torch:', torch.__version__, '| CUDA:', torch.cuda.is_available())
print('transformers:', transformers.__version__)
print('All OK!')
"
```

5. 下载模型权重

5.1 需要下载两个模型

模型	仓库	大小	存放路径
OpenVLA-7B 基础模型	openvla/openvla-7b	~15 GB	C:\vla-server\openvla-7b\
AeroVLA LoRA	XuPeng23/AerialVLA	~100 MB	C:\vla-server\openvla-7b\weight-lora\airial_vla\

路径必须完全匹配。 LoRA 必须在基础模型的 `weight-lora\airial_vla\` 子目录下。

5.2 开始下载

```
cd C:\vla-server

# 国内网络建议用镜像加速
$env:HF_ENDPOINT = "https://hf-mirror.com"

# 下载基础模型 (~15GB, 约 10-30 分钟)
python -c "
import os
os.environ['HF_ENDPOINT'] = 'https://hf-mirror.com'
from huggingface_hub import snapshot_download
snapshot_download('openvla/openvla-7b', local_dir=r'C:\vla-server\openvla-7b')
print('Base model done!')
"

# 下载 LoRA (~100MB, 约 30 秒)
python -c "
import os
os.environ['HF_ENDPOINT'] = 'https://hf-mirror.com'
from huggingface_hub import snapshot_download
snapshot_download('XuPeng23/AerialVLA', local_dir=r'C:\vla-server\openvla-7b\weight-lora\airial_vla')
print('LoRA done!')
"
```

5.3 验证文件完整性

```
# 关键文件必须存在
Test-Path C:\vla-server\openvla-7b\config.json
Test-Path C:\vla-server\openvla-7b\weight-lora\airial_vla\adapter_config.json
Test-Path C:\vla-server\openvla-7b\weight-lora\airial_vla\adapter_model.safetensors

# 应有多个 safetensors 分片文件
```

```
dir C:\vla-server\openvla-7b\model-*.safetensors
# 应列出 4 个分片文件
```

5.4 目录结构确认

下载完成后应该长这样：

```
C:\vla-server\
├─ openvla-7b\
│   ├── config.json
│   ├── model-00001-of-00004.safetensors  ← 基础模型权重
│   ├── model-00002-of-00004.safetensors
│   ├── model-00003-of-00004.safetensors
│   ├── model-00004-of-00004.safetensors
│   ├── preprocessor_config.json
│   ├── tokenizer.json
│   ├── tokenizer_config.json
│   └─ weight-lora\
│       └─ aerial_vla\
│           ├── adapter_config.json  ← LoRA 配置
│           └─ adapter_model.safetensors  ← LoRA 权重
├─ aerovla_server.py
└─ aerovla_inference.py
```

6. 启动推理服务

6.1 设置环境变量

```
$env:AEROVLA_BASE_MODEL = "C:\vla-server\openvla-7b"
$env:AEROVLA_LORA = "C:\vla-server\openvla-7b\weight-lora\aerial_vla"
$env:AEROVLA_PORT = "6006"
$env:AEROVLA_HOST = "0.0.0.0" # 监听所有网卡，允许局域网访问

# 强制离线模式 - 避免 transformers 尝试联网下载
$env:HF_HUB_OFFLINE = "1"
$env:TRANSFORMERS_OFFLINE = "1"
```

6.2 前台启动（测试，可看实时日志）

```
cd C:\vla-server
python aerovla_server.py
```

你应该看到：

```
=====
AeroVLA Remote Inference Server
Listening on http://0.0.0.0:6006
=====
[SERVER] Loading model from C:\vla-server\openvla-7b + LoRA ...
[AeroVLA] 4-bit: False, 8-bit: False
[SERVER] Model loaded and ready!
```

模型加载需要 **20-30 秒**。

6.3 验证服务

服务器本地测试：

```
Invoke-WebRequest -Uri http://localhost:6006/health | Select -Expand Content
# → {"status":"ok","model_loaded":true,"error":null}
```

笔记本端测试：

```
# 在笔记本 PowerShell 中
Invoke-WebRequest -Uri http://192.168.1.100:6006/health | Select -Expand Content
# 替换为你的服务器 IP
```

6.4 后台运行

确认前台测试没问题后，**Ctrl+C** 停掉，改为后台运行：

```
# 设置持久化环境变量（仅首次）
[Environment]::SetEnvironmentVariable("AEROVLA_BASE_MODEL", "C:\vla-server\openvla-7b", "Machine")
[Environment]::SetEnvironmentVariable("AEROVLA_LORA", "C:\vla-server\openvla-7b\weight-lora\airial_vla", "Machine")
[Environment]::SetEnvironmentVariable("AEROVLA_PORT", "6006", "Machine")
[Environment]::SetEnvironmentVariable("AEROVLA_HOST", "0.0.0.0", "Machine")
[Environment]::SetEnvironmentVariable("HF_HUB_OFFLINE", "1", "Machine")

# 后台启动（PowerShell 方式）
cd C:\vla-server
Start-Process python `
    -ArgumentList "aerovla_server.py" `
    -RedirectStandardOutput "server_stdout.log" `
    -RedirectStandardError "server_stderr.log" `
    -NoNewWindow

# 查看进程
Get-Process python | Where-Object { $_.MainWindowTitle -like "*aerovla*" }
```

```
# 查看日志
Get-Content server_stdout.log -Wait -Tail 20
```

6.5 注册为 Windows 服务（推荐生产环境）

使用 `nssm`（Non-Sucking Service Manager）将 Python 脚本注册为 Windows 服务：

```
# 1. 下载 nssm（一次性）
# https://nssm.cc/download → 解压 nssm.exe 到 C:\Windows\System32\

# 2. 创建服务
nssm install AeroVLA

# 弹出 GUI 配置窗口：
#   Application Path: C:\Users\管理员\AppData\Local\Programs\Python\Python310\python.exe
#   Startup Directory: C:\vla-server
#   Arguments: aerovla_server.py
#   点击 "Environment" 标签，添加：
#     AEROVLA_BASE_MODEL=C:\vla-server\openvla-7b
#     AEROVLA_LORA=C:\vla-server\openvla-7b\weight-lora\airial_vla
#     AEROVLA_PORT=6006
#     AEROVLA_HOST=0.0.0.0
#     HF_HUB_OFFLINE=1
#     TRANSFORMERS_OFFLINE=1

# 3. 启动服务
nssm start AeroVLA
nssm status AeroVLA
```

后续管理命令：

```
nssm start AeroVLA      # 启动
nssm stop AeroVLA       # 停止
nssm restart AeroVLA    # 重启
nssm status AeroVLA     # 查看状态

# 查看服务日志
Get-Content C:\vla-server\server_stdout.log -Wait -Tail 20
```

6.6 使用任务计划程序（备选方案）

如果不想装 `nssm`，也可以用 Windows 任务计划程序：

```
$action = New-ScheduledTaskAction -Execute "python" `
    -Argument "aerovla_server.py" `
```



```
-WorkingDirectory "C:\vla-server"

$trigger = New-ScheduledTaskTrigger -AtStartup

$principal = New-ScheduledTaskPrincipal -UserId "SYSTEM" -LogonType ServiceAccount

Register-ScheduledTask -TaskName "AeroVLA" `
    -Action $action -Trigger $trigger -Principal $principal `
    -Description "AeroVLA 推理服务" -RunLevel Highest

# 手动启动一次
Start-ScheduledTask -TaskName "AeroVLA"
```

7. 网络配置（局域网直连）

7.1 开放 Windows 防火墙

```
# 允许 6006 入站（PowerShell 管理员）
New-NetFirewallRule `
    -DisplayName "AeroVLA Inference (6006)" `
    -Direction Inbound `
    -Protocol TCP `
    -LocalPort 6006 `
    -Action Allow `
    -Profile Private,Public

# 验证规则
Get-NetFirewallRule -DisplayName "AeroVLA*" | Select DisplayName, Enabled,
Direction, Action
```

7.2 验证笔记本可以访问

```
# 在笔记本 PowerShell 中
Test-NetConnection 192.168.1.100 -Port 6006
# 应显示 TcpTestSucceeded : True

Invoke-WebRequest -Uri http://192.168.1.100:6006/health | Select -Expand Content
# 应返回 {"status":"ok",...}
```

如果 `Test-NetConnection` 失败：

- 检查服务器 Windows 防火墙是否开放了 6006
- 检查笔记本和服务器是否在同一局域网
- `ping 192.168.1.100` 是否能通
- 服务器上 `netstat -an | findstr 6006` 确认服务正在监听 `0.0.0.0:6006`

8. 本地 Windows 笔记本配置

8.1 修改 run_aerovla.bat

用文本编辑器打开 `d:\vla飞控部署\run_aerovla.bat`，修改 `SERVER_URL`：

```
REM 改为你的 Windows Server 局域网 IP
set SERVER_URL=http://192.168.1.100:6006
```

8.2 本地连通性测试

```
conda activate openvla
cd d:\vla飞控部署
python aerovla_client.py http://192.168.1.100:6006
```

预期输出：

```
Testing connection to http://192.168.1.100:6006...
Health: {'status': 'ok', 'model_loaded': True, 'error': None}
Result: {'fwd': 2.5, 'down': 0.0, 'yaw': 0.0, 'land': False, 'infer_time': 3.94}
```

看到这个就说明远程推理链路完全打通！

9. 端到端飞行测试

9.1 启动前自检

- ☐ 服务器上 `aerovla_server.py` 在后台运行（Windows 服务或 Start-Process）
- ☐ `Invoke-WebRequest http://服务器IP:6006/health` 返回 `ok`
- ☐ `run_aerovla.bat` 中 `SERVER_URL` 已改为服务器 IP
- ☐ AirSim settings.json 已配置好

9.2 启动（严格按顺序）

第一步：启动 AirSim 三件套

双击 `run_airsim.bat`，等待：

- AirSimNH 3D 窗口弹出并加载完成
- 控制器终端显示 `Starting main loop...`
- 摄像头终端显示 `Starting capture at 5 FPS`

第二步：启动 VLA GUI

双击 `run_aerovla.bat`，GUI 弹出后应看到：

```
[Remote] Connected to http://192.168.1.100:6006, model ready!  
[VLA] Connected to remote AeroVLA service!
```

第三步：起飞

GUI 中点击「起飞」，无人机飞到 8 米高度。

第四步：输入目标、启动 VLA 闭环

输入指令如 `a red car parked on the street`，点击「开始VLA闭环」或按回车。

日志区实时显示：

```
--- VLA Step 1/200 ---  
[Remote] 3.94s | fwd=2.50 down=0.00 yaw=0.150 land=False
```

第五步：降落

模型自动输出 `land=True` 时降落，或点击「降落」手动降落。

10. 性能预估

项目	局域网 Windows Server	云端 AutoDL (Linux)
网络延迟	<1 ms	30-200 ms
推理速度	~3-5 秒/步	~3-5 秒/步
带宽	千兆/万兆	取决于宽带
稳定性	极高	一般
bitsandbytes 兼容性	⚠ RTX 40 系不支持	Linux 完全兼容
推荐量化	bf16 全精度 (≥16GB)	8-bit (成本友好)
成本	一次性硬件投入	按小时付费

11. 后续每次启动（日常操作流程）

首次部署按照第 1-9 章走完后，以后每次使用时只需按以下步骤操作。**服务器端如果注册了 nssm 服务或任务计划程序，开机即自动运行，无需手动干预。**

11.1 检查清单（每次飞行前）

```
# === 在 Windows Server 上 ===  
  
# 1. 确认推理服务在运行
```

```
Invoke-WebRequest -Uri http://localhost:6006/health | Select -Expand Content
# 应返回 {"status":"ok","model_loaded":true,...}
# 如果返回 503 / Connection refused → 服务未启动，跳到 11.2

# 2. 确认 GPU 正常
nvidia-smi
# 应看到 python 进程占用 ~14.5GB 显存 (bf16) 或 ~8GB (8-bit)
```

11.2 如果推理服务没启动

```
# === 在 Windows Server 上 ===

# 方式 A: 如果注册了 nssm 服务
nssm status AeroVLA
# Stopped → nssm start AeroVLA
# 不存在 → 用方式 B 或 C

# 方式 B: Start-Process 后台启动
cd C:\vla-server
Start-Process python -ArgumentList "aerovla_server.py" `
    -RedirectStandardOutput "server_stdout.log" `
    -RedirectStandardError "server_stderr.log" `
    -NoNewWindow
# 等待 30 秒加载模型

# 方式 C: 如果用了任务计划程序
Start-ScheduledTask -TaskName "AeroVLA"
```

11.3 每次飞行的完整启动流程

步骤	在哪操作	操作	预计耗时
1	Windows Server	确认推理服务运行中 (<code>Invoke-WebRequest /health</code>)	5 秒
2	笔记本	双击 <code>run_airsim.bat</code>	30 秒
3	笔记本	等待 AirSimNH 3D 窗口加载完成	10-20 秒
4	笔记本	等待控制器终端显示 <code>Starting main loop...</code>	5 秒
5	笔记本	等待摄像头终端显示 <code>Starting capture at 5 FPS</code>	5 秒
6	笔记本	双击 <code>run_aerovla.bat</code> ，观察日志确认连接成功	5 秒
7	笔记本	GUI 中点击「起飞」	10 秒
8	笔记本	输入指令 → 点击「开始VLA闭环」→ 观察飞行	—

总耗时约 1-2 分钟（不含模型加载，模型通常在服务器开机时已加载完）。

11.4 飞行结束后

```
# === 在笔记本上 ===

# 1. 关闭 VLA GUI
# 2. 关闭 AirSimNH 窗口（按 Esc 或点 X）
# 3. 关闭控制器和摄像头的终端窗口

# === 在 Windows Server 上（可选） ===

# 保持推理服务运行，下次直接用。如果需要释放显存：
nssm stop AeroVLA
# 或
Stop-Process -Name python -Force
```

11.5 最快启动方式（推荐配置）

将以下配置到位后，每次只需 **双击两个 bat 文件**：

配置项	设置方法	效果
服务器 nssm 服务	按 6.5 节 配置	服务器开机自动启动推理，无需手动干预
笔记本 run_airsim.bat	已就绪	一键启动 AirSim 三件套
笔记本 run_aerovla.bat	已配置 SERVER_URL	一键启动 GUI

以后每次：

- 1. 开机 → 服务器自动拉起推理服务
- 2. 笔记本双击 run_airsim.bat → 等待加载
- 3. 笔记本双击 run_aerovla.bat → 起飞 → 飞！

12. WSL 方案评估

WSL (Windows Subsystem for Linux) 是否可以作为 GPU 服务器的替代方案？

12.1 结论：WSL 可以做推理，但有限制

能力	WSL2	裸 Windows Server	原生 Linux
CUDA 支持	支持	支持	支持
PyTorch GPU 训练/推理	支持	支持	支持
bitsandbytes	完全兼容，不卡死	RTX 40 系卡死	完全兼容
显存访问	与 Windows 共享	独占	独占
网络端口暴露	需额外配置	直接可用	直接可用
AirSim 仿真	不可用（无 GUI 渲染）	不可用	不可用（无头环境）
systemd 服务管理	WSL2 新版支持	nssm	systemd

能力	WSL2	裸 Windows Server	原生 Linux
性能开销	轻微 (~3-5%)	无	无

12.2 WSL 的杀手优势

WSL2 **特别适合 RTX 40 系列 GPU** 运行 VLA 推理，核心原因只有一个：

bitsandbytes 在 WSL2 内完全正常。 WSL2 内是真正的 Linux 内核，bnb 4-bit / 8-bit 均稳定运行，不会像裸 Windows 那样在 RTX 40 系（CC 8.9）上卡死。

这意味着：

- RTX 4070 12GB → WSL2 + 8-bit 量化 → **可用**
- RTX 4080 16GB → WSL2 + 8-bit 或裸 Windows bf16 → **都行**
- RTX 4090 24GB → WSL2 / 裸 Windows 全精度 → **都行**

12.3 WSL 推理服务器部署步骤

```
# === 1. 安装 WSL2（如果在 Windows Server 上） ===
# 在管理员 PowerShell 中：
wsl --install -d Ubuntu-22.04
wsl --set-default-version 2

# === 2. 进入 WSL，安装 CUDA Toolkit ===
# WSL2 内自动继承 Windows 的 NVIDIA 驱动，只需装 CUDA Toolkit：
wget https://developer.download.nvidia.com/compute/cuda/repos/wsl-ubuntu/x86_64/cuda-keyring_1.1-1_all.deb
sudo dpkg -i cuda-keyring_1.1-1_all.deb
sudo apt update
sudo apt install -y cuda-toolkit-12-1

# === 3. 安装 Python ===
sudo apt install -y python3 python3-pip

# === 4. 按原始 Linux 版指南操作 ===
# 创建目录、上传代码、安装依赖、下载模型
# 路径示例：/home/user/vla-server/
# pip3 install ... 版本号同本文档第 4 章

# === 5. 启动服务 ===
export AEROVLA_BASE_MODEL=/home/user/vla-server/opencvla-7b
export AEROVLA_LORA=/home/user/vla-server/opencvla-7b/weight-lora/aerial_vla
export AEROVLA_PORT=6006
export AEROVLA_HOST=0.0.0.0
nohup python3 aerovla_server.py > server.log 2>&1 &
```

12.4 WSL 网络配置（关键！）

WSL2 使用 NAT 网络，每次重启 IP 会变化，需要在 Windows 侧做端口转发：

```
# === 在 Windows Server 管理员 PowerShell 中 ===

# 1. 获取 WSL 的 IP
wsl hostname -I
# 例如: 172.25.100.50

# 2. 添加端口转发 (每次 WSL 重启后 IP 可能变化)
netsh interface portproxy add v4tov4 `
    listenport=6006 listenaddress=0.0.0.0 `
    connectport=6006 connectaddress=172.25.100.50

# 3. 防火墙放行
New-NetFirewallRule -DisplayName "AeroVLA WSL (6006)" `
    -Direction Inbound -Protocol TCP -LocalPort 6006 -Action Allow

# 4. 验证
Invoke-WebRequest -Uri http://localhost:6006/health
```

端口转发自动化脚本 (解决 IP 变化问题):

```
# 保存为 C:\vla-server\wsl-portforward.ps1
# 设为每次 WSL 启动后或 Windows 开机时运行

# 清除旧转发
netsh interface portproxy delete v4tov4 listenport=6006 listenaddress=0.0.0.0

# 获取 WSL IP
$wsl_ip = (wsl hostname -I).Trim()

# 添加新转发
netsh interface portproxy add v4tov4 `
    listenport=6006 listenaddress=0.0.0.0 `
    connectport=6006 connectaddress=$wsl_ip

Write-Host "Port forward: 0.0.0.0:6006 → $wsl_ip :6006"
```

将该脚本添加到任务计划程序 → 触发器设为 "计算机启动时"。

12.5 WSL vs 裸 Windows: 决策树

```
你的 GPU 是哪款?
├── RTX 4090 (24GB)
│   └── 🌈 裸 Windows 全精度 bf16, 零坑, 不需要看这一章
├── RTX 40 系列 (4080 / 4070 / 4060)
│   ├── 显存 ≥16GB (4080)
│   │   └── 裸 Windows 全精度 bf16 最省事
│   │       (WSL2 也可以, 但多一层转发没必要)
│   └── 显存 <16GB (4070 / 4060)
│       └── WSL2 全精度 bf16 最省事
```

└─ 显存 8-12GB (4060/4070)
 └─ **必须用 WSL2 + 8-bit 量化**
 (裸 Windows 上 bnb 4-bit 会卡死)

└─ RTX 30 系列 / A100 / V100
 └─ 裸 Windows 或 WSL2 都可以, bnb 兼容
 显存 ≥16GB 全精度, 否则 8-bit

└─ AMD GPU
 └─ 仅 WSL2 (ROCm 支持), 裸 Windows 不可用

一句话总结：显存 16GB 以上直接裸 Windows 全精度最省事；显存不足的 RTX 40 系必须用 WSL2 才能跑量化。

附录A：故障排查手册

依赖安装错误

错误信息	原因	解决
No module named 'torch'	PyTorch 未安装	<code>pip install torch torchvision</code>
is_offline_mode is not defined	huggingface_hub 版本太高	<code>pip install huggingface_hub==0.23.4</code>
cannot import AutoModelForVision2Seq	transformers 版本不对	<code>pip install transformers==4.42.4</code>
No module named 'peft'	peft 版本不对	<code>pip install peft==0.11.1</code>
bitsandbytes CUDA 报错	CUDA 版本和 torch 不匹配	重装匹配的 torch 版本
timm 版本错误	版本范围不对	<code>pip install "timm>=0.9.10,<1.0.0"</code>
numpy 兼容性错误	numpy 2.x 不兼容	<code>pip install "numpy<2.0.0"</code>
No module named 'bitsandbytes'	bnb 安装失败	检查 CUDA 版本；或直接不用 bnb 用全精度
bcrypt / passlib 错误	passlib 与 bcrypt 版本不兼容	代码已改用原生 bcrypt；重装： <code>pip install bcrypt</code>

模型加载错误

错误信息	原因	解决
config.json not found	AEROVLA_BASE_MODEL 路径不对	路径必须指向包含 config.json 的目录
adapter_config.json not found	LoRA 路径不对	LoRA 必须在基础模型下的 <code>weight-lora\Aerial_vla\</code>

错误信息	原因	解决
CUDA out of memory	显存不足	换更大显存 GPU
模型输出始终为 LAND	4-bit 量化退化	改用全精度 bf16 (load_in_8bit=False)
加载时卡住不动	bnb 4-bit 在 RTX 40 系卡死	改用全精度 bf16，不装 bitsandbytes
Params4bit / requires_grad_ RuntimeError	4-bit + peft 兼容性 bug	代码已内置 monkey-patch，确认 aerovla_inference.py 最新
加载时提示下载文件	HF_HUB_OFFLINE 没设	设 \$env:HF_HUB_OFFLINE = "1"

网络连接错误

错误信息	原因	解决
Connection refused	服务没启动或端口没监听在 0.0.0.0	netstat -an findstr 6006 检查；确认 AEROVLA_HOST=0.0.0.0
Connection timed out	Windows 防火墙阻断	New-NetFirewallRule 放行 6006
能 ping 通但端口不通	防火墙只放行了 ICMP	添加 6006 端口的入站规则
503 Service Unavailable	模型还没加载完	等 30 秒再试
Test-NetConnection 成功但 curl 失败	curl 语法或代理问题	用 Invoke-WebRequest 测试

运行时错误

错误	原因	解决
推理返回 500 + "Failed to decode"	图像数据损坏	检查客户端是否正确编码 JPEG
推理时间异常长 (>30s)	GPU 上可能跑了其他任务	nvidia-smi 查看 GPU 利用率
进程突然消失	内存不足被系统杀掉	任务管理器查看内存；增加虚拟内存
服务运行但 GPU 没被用	PyTorch 用了 CPU 而非 CUDA	python -c "import torch; print(torch.cuda.is_available())"

附录B：日常运维命令速查

```
# ===== 服务管理 =====
Get-Process python | Where-Object { $_.Path -like "*aerovla*" } # 查看进程
nssm status AeroVLA # nssm 服务状态
```

```
nssm restart AeroVLA # 重启服务
Stop-Process -Name python -Force # 强制停止

# ===== 日志 =====
Get-Content C:\vla-server\server_stdout.log -Wait -Tail 20 # 实时日志
Get-Content C:\vla-server\server_stdout.log -Tail 100 # 最近100行
Clear-Content C:\vla-server\server_stdout.log # 清空日志

# ===== GPU =====
nvidia-smi # GPU 状态
nvidia-smi -l 1 # 每秒刷新
nvidia-smi --query-gpu=memory.used --format=csv # 只看显存

# ===== 端口 =====
netstat -an | findstr 6006 # 查看端口监听
Get-NetTCPConnection -LocalPort 6006 # 查看端口被谁占用

# ===== 一键重启 =====
Stop-Process -Name python -Force
Start-Sleep 2
cd C:\vla-server
Start-Process python -ArgumentList "aerovla_server.py" `
    -RedirectStandardOutput "server_stdout.log" `
    -RedirectStandardError "server_stderr.log" `
    -NoNewWindow
Write-Host "Restarted. Waiting 30s for model to load..."
Start-Sleep 30
Invoke-WebRequest -Uri http://localhost:6006/health | Select -Expand Content
```

附录C：全精度 bf16 部署（推荐 $\geq 16\text{GB}$ 显卡）

如果你的 GPU 有 $\geq 16\text{GB}$ 显存（RTX 3090 / 4080 / 4090），强烈推荐全精度 bf16，完全避开 bitsandbytes 兼容性问题。

编辑 `aerovla_server.py`，找到 `AeroVLAInference` 初始化部分，确认：

```
_engine = AeroVLAInference(
    base_model_path=base_path,
    lora_path=lora_path,
    load_in_4bit=False, # 必须 False
    load_in_8bit=False, # 必须 False → 全精度 bf16
    log_fn=log_fn,
)
```

重启服务即可。全精度下显存占用 $\sim 14.5\text{ GB}$ 。

附录D：Windows Server 与 Linux Server 的关键差异

项目	Linux Server	Windows Server
模型路径	/root/vla-deploy/	C:\vla-server\
环境变量	export KEY=VAL	\$env:KEY = "VAL"
后台运行	nohup ... &	Start-Process ...
开机自启	systemd	nssm / 任务计划程序
防火墙	ufw allow	New-NetFirewallRule
进程查看	ps aux	Get-Process
端口监听	ss -tlnp	netstat -an
bitsandbytes 4-bit	稳定	卡死 (RTX 40 系)
bitsandbytes 8-bit	稳定	不稳定
推荐量化	8-bit	bf16 全精度
scp 上传	支持	支持 (Windows 内置)

附录E：安全加固建议

- 1. **6006 端口仅监听局域网**：如果不需外网访问，改 `AEROVLA_HOST=127.0.0.1`，通过 SSH 隧道访问
- 2. **Windows 防火墙限制 IP**：

```
New-NetFirewallRule -DisplayName "AeroVLA (Restricted)" `
    -Direction Inbound -Protocol TCP -LocalPort 6006 `
    -Action Allow -RemoteAddress 192.168.1.50/32
```

- 3. **定期备份模型**：LoRA 权重 100MB 很小，基础模型 15GB 可随时重下
- 4. **Windows Update**：生产服务器建议关闭自动重启，避免训练/推理中断

文档版本: 2026-07-09 (Windows Server 版)